INF201 - Lecture 8

26th Oct 2026

Topics

- Organising your code
- Object-oriented programming

Organising your code

When you work on projects, it is a good idea to separate between:

1. Reusable code - in Python scripts
2. Interactive analysis or visualisation - in Jupyter Notebooks
3. Documentation - in README.md and Jupyter Notebooks

Why work like this?

- Makes it easier to re-use parts of the code for this or other projects
    - We can for example import a function from another python file by `from my_analysis_code import summary_statistics`
- Cleaner git diffs
    - One-liners in `.py` files
    - Includes all metadata in `.ipynb` files
    - Consider using the nbstripout package to "clean up" your notebooks before committing them to git
- Here are a couple of excellent resources on the topic:
    - The Good Research Handbook
    - Good enough practices in scientific computing

Your project could be organised something like this:

```
    project_name/
    ├── .gitignore
    ├── README.md
    ├── requirements.txt           # info about your venv
    ├── data/                      # consider adding this folder to
.gitignore
    │   ├── raw/
    │   └── processed/
    ├── notebooks/                 # your jupyter notebook(s)
    │   ├── 01_data_cleaning.ipynb
    │   └── 02_model_evaluation.ipynb
    ├── src/                       # python scripts with the majority
```

```
of your code
    │   ├── data.py               # functions for loading/transforming
data
    │   └── processing.py         # functions, classes, processing
data, ...
    └── tests/                    # folder for testing (more about
this later)
        └── test_processing.py
```

- An `src` folder typically contains the resusable logic for the project
  - functions
  - classes
  - data pipelines
- A `notebooks` or `scripts` folder contains single purpose scripts/notebooks
  - These will import code from `src`
  - The scripts/notebooks here are run directly, unlike scripts in `src`

A recap: How to write good code

- Style
  - Follow PEP-8
  - This includes sensible names for variables, functions and classes
- Functions
  - If you do something more than once, it should be a function!
  - Many small functions are better than few large functions
  - I recommend writing all functions at the beginning of your scripts
  - Use doc-strings and type hints
  - A script with several functions will typically have a `main()` function that runs when the script itself runs,
    but not when it is imported to other scripts
    ```python
    if __name__ == "__main__":
        main()
    ```
- Efficency
  - Chose appropriate data containers
  - Vectorise instead of using loops if possible
- Structure and reproducibility
  - Use virtual environments
  - Organise your data, code and notebooks
  - Use version control (like Git)

Let's make a game!

- We want to create an RPG in Python
  - The players are characters on an adventure
  - They get into battles
  - They find items

- The script `game.simulator_v0.py` contains a couple of useful functions, let's have a look
  - In Jypter notebook, you can use `?` to get information on a function or `??` to show the code itself as well

In [ ]:
```
import game_simulator_v0 as g0

g0.take_damage?
```

In [ ]:
```
g0.take_damage??
```

In [ ]:
```
g0.heal??
```

In [ ]:
```
# Introduce a player
player1 = {"name": "Aria", "hp": 100, "inventory": ["potion", "shield"]}

# The player can be injured
g0.take_damage(player1, 10)
print(f"{player1["name"]} is injured and now has {player1["hp"]} hit points")

# The player can be healed
g0.heal(player1, 5)
print(f"{player1["name"]} heals and now has {player1["hp"]} hit points")
```

- What happens if we have several players?
- How do we make sure that HP stays within allowed boundaries?
- How do we keep track of which functions belong to which entities?
  - Which actions should players be able to do?
  - What if different players have different available actions?

In [ ]:
```
# Nothing is preventing us from setting HP to "illegal" values
player1["hp"] = -999
player1
```

- We will look at an alternative approach for this task

Object-oriented programming

- Mini intro to object-oriented Python
- Object-orented programming - Wikipedia article
- This is one of several programming paradigmes:
  - Procedural: Procedures (e.g. functions) calling each other
  - Object-oriented: Objects containing both data and associated methods
- The first object-orented programming language was developed in Norway!
  - Simula
  - Created by Ole-Johan Dahl and Kristen Nygaard in the 1960s

Aspect	Procedural Approach	Object-Oriented Approach
Core Focus	Functions acting on standalone data (`fn(data)`)	Objects bundling data and behavior (`object.method()`)
Data Organization	Scattered variables and containers (`player["hp"]`)	Structured attributes defined in a custom class (`player.hp`)
State Protection	Data is exposed: any function can mutate it accidentally	Encapsulated: state is protected and updated via explicit methods
Code Structure	Sequential step-by-step logic and utility scripts	Modular, self-contained components with clear public interfaces
IDE Ergonomics	Limited autocomplete: prone to typos in dictionary keys	Full VS Code IntelliSense, real-time type checking, and inline docs
Best Used For	Quick scripts, data pipelines, and linear data transformations	Large codebases, complex state management, and team collaboration

> Python is an object-oriented programming language, and in Python everything is an object.

The Whirlwind Tour of Python by Jake VanderPlas

- Good news: You already use object-oriented programming
- We look at `numpy` as an example

In [ ]:
```
import numpy as np

ones_array = np.ones((2, 3), dtype=np.int8)
ones_array
```

In [ ]:
```
np.ones?
```

In [ ]:
```
print(f"Is ones_array an object? {isinstance(ones_array, object)}")
```

In [ ]:
```
print(f"Type: {type(ones_array)}")
```

In [ ]:
```
print(f"Data type attribute: {ones_array.dtype}")
print(f"Shape attribute: {ones_array.shape}")
```

In [ ]:
```
print("Methods:")
print(dir(ones_array))
```

In [ ]:
```
ones_array.sum()
# ones_array.sum(dtype=np.int8)
```

Object-oriented concepts

- **Class**: A template for creating an object
- **Object**: An instance of a class
- **Attribute**: Variables that belong to a class
- **Method**: Function that operates on objects of the class

Creating a Class

- Created by `class` in Python
- Similar to creating functions
- Should also use doc-strings and type hints
- Class names use `CamelCase`
- `example_module.py` has a PEP8 compliant template for writing classes

In [ ]:
```python
from example_module import ClassName

obj = ClassName(param1="Hello", param2="World")
print(obj.method_name("Test"))
print(ClassName.utility_method("Check"))
default_obj = ClassName.create_default()
print(default_obj.method_name("Default Test"))
```

- Let's make a character class for our RPG game
  - We want the character to be able to fight
- Which attributes does the character need?
  - Name
  - Health (hp), both current and max/min
  - Damage, i.e. how much damage does the character deal when attacking?
- What should the character be able to do (i.e. methods)?
  - Attack

1. Set up the class with a name and a doc-string

In [ ]:
```python
class Character:
    """Represents a character in the RPG"""
```

2. Initialise using `__init__` and `self`
   - The `__init__` method is the *constructor* for this class
   - It is used to create a specific object of this class
   - `self` is a placeholder saying "belonging to whichever specific object is currently running this code"
   - We use type hints for the attributes

In [ ]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
```

```
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage
```

3. Add a method that allows the character to attack
    - This method is linked to `self` and a differnt object (here called `target`)

In [ ]:
```
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target: Character) -> None:
        """Attack and deal damage to target Character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0
```

We can now create two characters and have them attack each other

In [ ]:
```
character1 = Character(name="Aria", hp_max=100, damage=10)
character2 = Character(name="Kura", hp_max=100, damage=20)

print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

In [ ]:
```
character1.attack(character2)
print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

In [ ]:
```
character2.attack(character1)
print(f"{character1.name} has {character1.hp} out of {character1.hp_max} left")
print(f"{character2.name} has {character2.hp} out of {character2.hp_max} left")
```

- Let's talk about `self` again here
- Python reads `character1.attack(character2)` as `Character.attack(character1, character2)`
- The dot notation is *syntactic sugar* where the instance to the left is automatically passed as the first argument of the function
- If you forget `self` in the method definition, you will run into trouble

In [ ]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # ASsume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0

    def introduce() -> None:  # Here we forgot to include self!
        """Creates string that introduces character"""
        print(f"Hi, my name is {name}")
```

In [ ]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.introduce()
```

In [ ]:
```python
Character.introduce(character1)
```

In [ ]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self.hp = hp_max  # Assume the character is created with max hp
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target.hp -= self.damage
        target.hp = max(target.hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")
```

In [ ]:
```
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.introduce()
```

4. Make objects inspectable using `__repr__` and `__str__`
   - `__repr__`: For developers and debuggers. Should ideally look like valid Python code to recreate the object, or a unambiguous string.
   - `__str__`: For end users and clean outputs (`print()` statement)

In [ ]:
```
character1  # will call __repr__
```

In [ ]:
```
print(character1)  # will call __str__
```

In [ ]:
```
ones_array  # __repr__ with more useful output
```

In [ ]:
```
print(ones_array)  # __str__ with more useful output
```

In [ ]:
```
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self._hp = hp_max  # Note: _hp indicated private attribute
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target._hp -= self.damage
        target._hp = max(target._hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.damage}>)

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damage} dama
```

In [ ]:
```
character1 = Character(name="Aria", hp_max=100, damage=10)
character1  # will call __repr__()
```

In [ ]:
```
print(character1)  # will call __str__
```

5. Protect internal state (encapsulation using `@property`)
    - We want to prevent external code to setting class attributes to invalid values
        - Example: Keep `hp` between `0` and `hp_max`
    - For this we start the attribute name with a single underscore `_`
    - This signals that this attribute should be considered protected (internal data), meaning it should only be accessed within the class (or by subclasses)
    - This is a naming convention, and will not be enforced
    - By also including `@property`, we can enforce this

In [ ]:
```python
class Character:
    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10) -> None:
        """
        Initialise the Character

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage delt when character attacks (default = 10)
        """
        self.name = name
        self._hp = hp_max  # Note: _hp indicated private attribute
        self.hp_max = hp_max
        self.damage = damage

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        target._hp -= self.damage
        target._hp = max(target._hp, 0)  # Target hp cannot end up lower than 0

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.damage}>)

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damage} dama

    @property
    def hp(self) -> int:
        """Public read-only property to inspect HP without allowing direct, unsa
        return self._hp
```

In [ ]:
```python
character1 = Character(name="Aria", hp_max=100, damage=10)
character1.hp
```

In [ ]:
```python
character1.hp = 50
```

In [ ]:
```python
character1._hp
```

In [ ]:
```
character1._hp = 50
```

In [ ]:
```
character1._hp
```

- `_hp` is the private container holding the raw value
- `@property` is the gatekeeper controlling how the outside world views or changes that value.

What does @ mean here?

- `@` methods in Python are called decorators
- A decorator is syntactic sugar that wraps a function or method to modify or extend its behavior without altering its underlying code
- `@property` turns a method into an attribute with hidden logic
- In our example `@property` creates an attribute `hp` that can be read but not changed (until we define a method for it)
- We can therefore view it by `character1.hp`, but not change it
- However, we can still change the attribute `_hp`
- Think of `hp` as the front door where the general audience will view and access the attribute
  while `_hp` is the backdoor where developer can get access

Summary of our Character class so far

- We have create a class with an initialiser
- Objects of this class has attributes
  - `name`
  - `_hp`
  - `hp_max`
  - `damage`
- And a property attribute
  - `hp` - protecting `hp`
- Objects of this class have some regular methods
  - `attack()` to attack another object
  - `introduce()` to introduce themselves
- ... and some special methods
  - `__repr__` for representation
  - `__str__` for display representation

We might also want

- Sub-classes for different types of characters
- Methods representing other actions
- Methods for generating default characters
- Different classes, for example Dice
- Inventories with actions depending on inventory items